Estudio y Plan Técnico Final v2.2 — WhatsApp Multi-Proveedor

PLAN FINAL v2.2

Fecha: 20/10/2026
Estado: Sin cambios en código
Gestor paquetes: `pnpm`

YCloud + Evolution API v2.3.7 + Meta + Pausa Global + Human Handoff + Envío Manual Admin UI & CLI DGX

Proyecto: **CRM Salvadora Conesa** (`D:\tmp\antigraviti\crm_salvadora`)
Evolution API en DGX Spark: `https://evolutionapi.jigretera.com` (v2.3.7) · Número chatbot: **+34 646 034 231**

Resumen de Funcionalidades y Correcciones en el Plan Final v2.2

- **Nuevo en v2.2 — Pausa Global de Automatización de WhatsApp ("Kill Switch" sin desconectar sesión):** Persistido en `AppSettings.whatsappAutomationPaused` (tabla `app_settings`). Sigue recibiendo y guardando mensajes en Inbox, permite respuestas manuales desde Inbox, utilidad Admin UI y CLI Python, pero detiene `AgentRunnerService` , recordatorios automáticos y notificaciones automáticas de citas/eventos/OTP. Registra `whatsapp.automation_paused` y `whatsapp.automation_resumed` en `audit_logs` .
- **Nuevo en v2.2 — Conservación e Independencia Total de Human Handoff por Conversación:** Se preserva la implementación actual (`conversations.handoff` + `PATCH /api/conversations/:threadId/handoff`) funcionando de forma idéntica con YCloud, Evolution API y Meta Cloud API, sin que la pausa global altere ni pierda el estado individual de handoff de cada hilo y sin contestar retroactivamente mensajes acumulados al reanudar.
- **Correcciones v2.1 mantenidas:** Envío binario `multipart/form-data` en UI y CLI (conversión a Base64 exclusivamente dentro de `EvolutionProvider`), validación profunda de estructura interna ZIP para `.docx` (`word/` + `[Content_Types].xml`) y `.xlsx` (`xl/` + `[Content_Types].xml`), red interna Docker Swarm verificada en el DGX Spark, y consulta en tiempo real del estado de Evolution API.

PARTE I — PAUSA GLOBAL DE AUTOMATIZACIÓN Y HUMAN HANDOFF POR CONVERSACIÓN (v2.2)

1. Persistencia del Flag Global de Automatización y Servicios que lo comprueban

El flag global se almacena en la entidad existente `AppSettings` (tabla singleton `app_settings` en `backend/src/common/entities/app-settings.entity.ts`), nunca en una variable de entorno:

```
// En AppSettings (tabla app_settings):
@Column({ type: 'boolean', default: false })
whatsappAutomationPaused: boolean; // false = ACTIVO (por defecto), true = PAUSADO
```

Servicio / Flujo en Backend	Tipo de Operación	Comportamiento cuando `whatsappAutomationPaused === true`
`YCloudWebhookController` `EvolutionWebhookController` `AgentRunnerService.run()`	**Entrada + Respuesta Automática IA**	1) **SÍ recibe** el webhook y actualiza contacto. 2) **SÍ guarda** el mensaje entrante en `messages` y actualiza `conversations` (`unreadCount + 1`). 3) **SÍ emite** SSE `message.received` para mostrarlo al instante en el Inbox. 4) **NO ejecuta** `AgentRunnerService` / Mastra LLM ni envía ninguna respuesta automática.
`RemindersService.sendReminder()`	**Recordatorios Automáticos (Cron)**	Comprueba `settingsService.isWhatsappAutomationPaused()` : **NO envía** recordatorios automáticos por WhatsApp mientras esté pausado.
`AppointmentsService` `ServicesService` `EventEditionsService` `ContactVerificationService`	**Notificaciones Automáticas (Citas, Eventos, Quórum, OTP)**	Comprueban `settingsService.isWhatsappAutomationPaused()` (además de barrera defensiva en `WhatsappGatewayService.sendAutomatedMessage()`): **NO envían** mensajes automáticos de WhatsApp.
`ConversationsController.sendManual()`	**Respuesta Manual del Operador desde Inbox**	**SÍ FUNCIONA:** Llama a `WhatsappGatewayService.sendManualMessage()` y entrega el mensaje por el proveedor activo del agente (YCloud, Evolution o Meta).
`WhatsappAdminSendService.send()` (UI Admin `/settings` y CLI Python `tools/whatsapp_send.py`)	**Envío Manual Admin / CLI Python en DGX**	**SÍ FUNCIONA:** Al ser un envío manual explícito, ignora la pausa de automatización y envía texto/archivos con normalidad.

2. Auditoría del Human Handoff existente y su reutilización multi-proveedor

Se ha auditado la implementación actual de Human Handoff en el CRM y se conserva al 100 % sin duplicar estructuras:

- **Persistencia actual:** Columna booleana `handoff` (`default: false`) en la entidad `Conversation` (tabla `conversations`, clave primaria `threadId = "${agentKey}:${phone}"`).
- **Endpoint actual:** `PATCH /api/conversations/:threadId/handoff` con body `{ handoff: boolean }` en `ConversationsController.setHandoff` (línea 194), que llama a `MessagesService.setHandoff(threadId, dto.handoff)` y emite el evento SSE `conversation.updated`.
- **Lógica actual en `AgentRunnerService.run()` (líneas 135–262):**
  1. Guarda primero el mensaje entrante con `saveInboundOnce` (que ejecuta `touchConversation` incrementando `unreadCount` sin alterar `conversations.handoff`) y emite SSE `message.received`.
  2. En las líneas 252–262 consulta `messagesService.getConversation(threadId)` ; si `conversation?.handoff === true` , registra en log `Thread is in human handoff — auto-reply skipped` y devuelve `{ reply: '', outbound: null }` sin llamar al LLM.
- **Desacoplamiento del envío manual del operador (`POST /api/conversations/:threadId/messages`):** En `ConversationsController.sendManual` (línea 269) se sustituye la llamada directa a `this.ycloudClient.sendTextMessage(...)` por `this.whatsappGatewayService.sendManualText(...)` . Así, cuando el operador responde manualmente en una conversación, el mensaje sale por **Evolution API** si el agente tiene `whatsappProvider === 'evolution'` , por **YCloud** si tiene `'ycloud'` , o por **Meta** cuando se active.

3. Interacción entre Pausa Global y Human Handoff + Garantía de No-Retroactividad

```
Mensaje Entrante (YCloud / Evolution / Meta)
       |
       ▼
1. Guardar en BD (messages + conversations.unreadCount++) y emitir SSE 'message.received' al Inbox
       |
       ▼
2. ¿AppSettings.whatsappAutomationPaused === true?  —[SÍ]—> FIN (No ejecuta AgentRunner, conserva conversation.ha
       | [NO]
       ▼
3. ¿Conversation.handoff === true (Modo Manual)?    —[SÍ]—> FIN (No ejecuta AgentRunner, operador atiende desde I
       | [NO]
       ▼
4. Ejecutar AgentRunnerService (Mastra/LLM) —> Respuesta automática —> WhatsappGatewayService (YCloud | Evolution
```

Independencia de estados y confirmación de NO-respuesta retroactiva al reanudar:

- **No se pierde el estado individual de Human Handoff:** `app_settings.whatsappAutomationPaused` y `conversations.handoff` son campos independientes. Pausar o reanudar la automatización global **jamás modifica** las filas de `conversations`. Al reanudar la automatización global, los hilos que tenían `handoff = false` vuelven a automático y los que tenían `handoff = true` siguen en manual hasta que un operador pulse *Reanudar agente*.
- **Confirmación de NO respuesta retroactiva:** El CRM procesa el agente únicamente en el instante en que entra el webhook de un mensaje nuevo; **no existe ningún worker ni cola que re-ejecute el LLM sobre mensajes entrantes antiguos** ya guardados con `status = 'received'` (y si un proveedor reintentara un webhook antiguo, `saveInboundOnce` lo descarta por su restricción única sobre `externalId`). Por tanto, al pasar de `PAUSADO` a `ACTIVO`, **nunca se contestarán retroactivamente** los mensajes acumulados durante la pausa.

4. Cambios en la UI de Ajustes, en el Inbox y Nuevos Eventos de Auditoría

Cambios en Ajustes (`/settings` — Solo Admin)

- **Control Global de Automatización de WhatsApp:** Tarjeta destacada `[ WhatsApp automático: ACTIVO / PAUSADO ]` con badge visual y botón de activación/pausa instantánea.
- Aclara explícitamente en la UI que pausar la automatización **no desconecta** Evolution API ni cierra la sesión de WhatsApp: los mensajes siguen entrando al Inbox y los envíos manuales siguen activos.
- **Utilidad de Envío Manual de WhatsApp (Admin):** Selector de agente, teléfono destino, texto opcional y 1 archivo opcional (`.docx, .pdf, .txt, .mp3, .mp4, .xlsx`) vía `FormData`.

Cambios en el Inbox (`/conversations`) y Auditoría

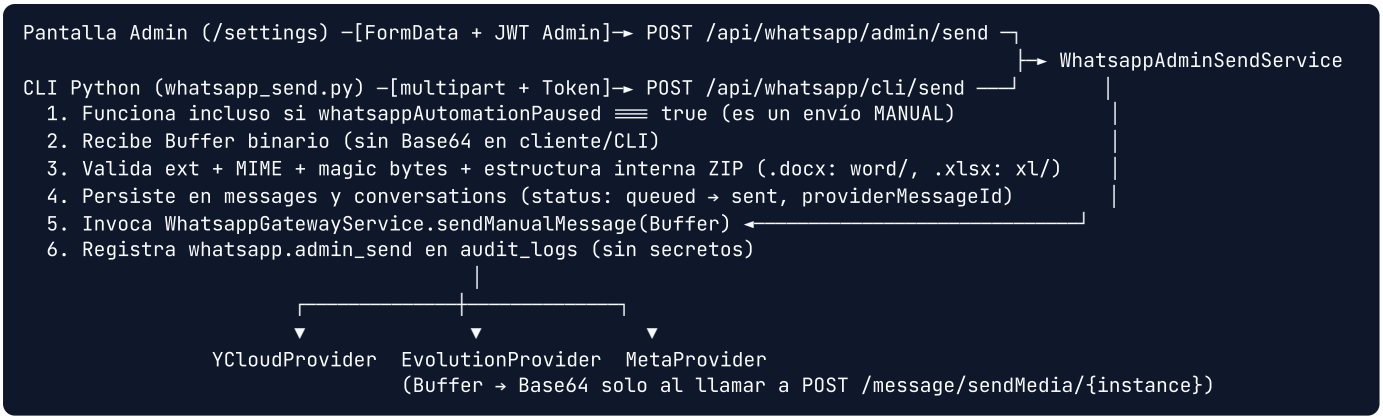
- **Banner de Pausa Global en Inbox:** Si `whatsappAutomationPaused === true`, muestra un aviso superior en el Inbox indicando que las respuestas automáticas están pausadas globalmente y que se puede responder manualmente.
- **Controles por conversación:** Botón claro en cabecera (*"Pausar agente / Tomar conversación"* ↔ *"Reanudar agente"*) y badge en la lista de hilos.
- **Nuevos eventos en `audit_logs` (`AuditAction`):**
  - `whatsapp.automation_paused` (con admin, IP y fecha)
  - `whatsapp.automation_resumed` (con admin, IP y fecha)
  - `whatsapp.admin_send` (envíos manuales UI Admin y CLI)

PARTE II — ESPECIFICACIÓN TÉCNICA CONSOLIDADA (v2.0 + v2.1 + v2.2)

5. Campos nuevos en Base de Datos (Migración aditiva sin pérdida de datos)

Tabla.Columna	Tipo SQL	Protección Write-Only	Propósito y Valor por Defecto
`app_settings.whatsappAutomationPaused`	`boolean` NOT NULL	Público en sesión CRM	`DEFAULT false` Pausa global ("kill switch") de las automatizaciones de WhatsApp.
`agent_configs.whatsappProvider`	`varchar(20)` NOT NULL	Público en admin	`DEFAULT 'ycloud'` Valores: `'ycloud' | 'evolution' | 'meta'`.
`agent_configs.evolutionApiUrl` `agent_configs.evolutionInstanceName`	`varchar` NULL	Público en admin	URL de Evolution API (ej. `https://evolutionapi.jigretera.com`) y nombre de instancia.
`agent_configs.evolutionApiKey` `agent_configs.evolutionWebhookSecret`	`varchar` NULL	SECRETO → `hasEvolution*`	API Key de Evolution y secreto fail-closed para cabecera `x-evolution-secret`.
`agent_configs.metaPhoneNumberId` `agent_configs.metaWabaId` `agent_configs.metaGraphVersion`	`varchar` NULL	Público en admin	Identificadores preparados para Meta WhatsApp Cloud API (default `v21.0`).
`agent_configs.metaAccessToken` `agent_configs.metaAppSecret` `agent_configs.metaVerifyToken`	`varchar` NULL	SECRETO → `hasMeta*`	Secretos preparados para Meta Cloud API, filtrados por `sanitizeAgentConfig`.

6. Envío Manual Multipart, Validación ZIP (`.docx` / `.xlsx`) y Evolution API v2.3.7



Formato	Validación Extensión + MIME + Estructura ZIP	Endpoint y Payload Verificado en Evolution API v2.3.7
Texto	Texto UTF-8 no vacío (si no hay archivo).	`POST /message/sendText/{instance}` → `{ "number": "34600111222", "text": "..." }`
`.docx`	MIME Word + firma `PK\x03\x04` + entradas internas ZIP `[Content_Types].xml` y `word/` (`word/document.xml`).	`POST /message/sendMedia/{instance}` → `{ "number", "mediatype": "document", "mimetype", "caption", "media": "<BASE64_PURO>", "fileName": "archivo.docx" }`
`.xlsx`	MIME Excel + firma `PK\x03\x04` + entradas internas ZIP `[Content_Types].xml` y `xl/` (`xl/workbook.xml`).	`POST /message/sendMedia/{instance}` → `{ "number", "mediatype": "document", "mimetype", "caption", "media": "<BASE64_PURO>", "fileName": "archivo.xlsx" }`
`.pdf` / `.txt`	`.pdf` : firma `%PDF-` . `.txt` : texto sin bytes nulos `0x00` .	`POST /message/sendMedia/{instance}` con `"mediatype": "document"` y `"fileName"` obligatorio.
`.mp4`	`video/mp4` + átomo `ftyp` en bytes 4..7.	`POST /message/sendMedia/{instance}` con `"mediatype": "video"` , `"mimetype": "video/mp4"` y `"caption"` opcional.
`.mp3`	`audio/mpeg` + cabecera `ID3` o sync frame MPEG.	`POST /message/sendMedia/{instance}` con `"mediatype": "audio"` . Si lleva texto + `.mp3` , envía primero `/message/sendText` y luego `/message/sendMedia` .

7. Red Interna Verificada en el DGX Spark y Suite de Tests a Añadir

- **Red Docker Swarm (`dokploy-network`) verificada en vivo en el DGX Spark (`192.168.1.17`):** El servicio Swarm estable del CRM es `ccmfallaviajeasevilla-crmsalvadoraconesa-34p307` (puerto `3001`) y el de Evolution API tiene el alias `evolution-api` (puerto `8080`). No se hardcodea ningún hostname en código: se usa `EVOLUTION_WEBHOOK_BASE_URL` (con fallback a `APP_URL`).
- **Estado de Evolution en tiempo real:** Consultado siempre contra `GET /instance/connectionState/{instance}` .
- **Suite de Tests Unitarios e Integración (Jest):**
  1. `whatsapp-gateway.service.spec.ts` : Enrutamiento por proveedor (`ycloud` , `evolution` , `meta`), bloqueo de envíos automáticos cuando `whatsappAutomationPaused === true` y permiso de envíos manuales durante la pausa.
  2. `agent-runner.pause-handoff.spec.ts` : Verificación de que con `whatsappAutomationPaused === true` o con `conversation.handoff === true` el mensaje entrante se guarda en BD y emite SSE al Inbox pero NO ejecuta el LLM ni envía respuesta automática; verificación de que pausar/reanudar globalmente conserva `conversation.handoff` intacto y no dispara respuestas retroactivas.
  3. `settings.whatsapp-pause-audit.spec.ts` : Emisión de `whatsapp.automation_paused` y `whatsapp.automation_resumed` en `audit_logs` con el actor administrador.
  4. `evolution.provider.spec.ts` : Conversión de `Buffer` a Base64 puro solo en `EvolutionProvider` , envíos de texto, documentos, vídeo y texto + audio `.mp3` .
  5. `evolution-webhook.controller.spec.ts` : Validación fail-closed de `x-evolution-secret` , deduplicación y mapeo de estados.
  6. `whatsapp-admin-send.service.spec.ts` : Recepción `multipart/form-data` como `Buffer` , rechazo de un `.zip` arbitrario renombrado a `.docx` o `.xlsx` , aceptación de `.docx` / `.xlsx` reales con `word/` o `xl/` , persistencia de `providerMessageId` y auditoría `whatsapp.admin_send` .
  7. `whatsapp-cli-token.guard.spec.ts` y `agents-config.sanitize.spec.ts` .